1.4 Report

1. The stack contains functions in memory as they are executed.

2. The program counter goes through instructions and represents the progress of the program's execution. I used GDB to locate the program counter by compiling stack.c with the debugging flag (-g), which then allowed me to see the line "read_only_string[0] = 'X';" along with it's program counter with the number in the aligator brackets.

3. to get the desired results for this part of the assignment, the program counter was adjusted manually to skip the instrcution that caused the segmentation fault.

2.2 Report

first_clear_bit - First, we compute the log_2 of the number we are given. Just like the log of a number in base 10 log_10, this gives us the number of digits in the number. Now that we have that, we can create a "mask" that will be a string of 1s with the length that we computed before.

After getting that mask, we can take the inverse of the number we got and apply the mask on that inverse. This gives us a number with a number of bits that is equivalent to the location of the first 0 from the MSB when counted from the LSB. and just like we did before, we can return the length of that number by calculating the log_2 of it.

set_bit_at_index - since we know that the bitmap is divided into bytes (techincally char's. but it's exactly the same), we know that to get the correct bitmap index, we can divide the index by the number of bits in a byte (8), and use the mod function to get the correct bit withing the byte by doing index%8 and setting that bit to 1 in the bit map

get_bit_at_index - exact same logic as set_bit_at_index, but instead of setting that bit to 1, we will simply retrieve that bit.